Fabian Williams Braintrust End-to-End Lab

From Basic Agents to Evals and Observability

GitHub Repo: https://go.fabswill.com/braintrustdeepdive

Ensure your FIRST STOP is in the ReadMe files!!!! Then come back to this guide for the WHY, and the WHAT, the GitHub is soley the HOW to do it.

Braintrust is an **end-to-end platform for building AI applications** that makes LLM development more robust and iterative[1]. In this hands-on lab, we'll start with a simple AI agent and progressively introduce **evaluations (evals)** and **observability** tools, going from the basics to advanced use cases. Each step includes **why it's important**, the **expected results**, and tips for **troubleshooting** potential issues. By the end, you will have an end-to-end workflow – from building and testing an AI agent, to instrumenting it with OpenTelemetry and integrating evaluation checks into CI/CD.

Prerequisites and Setup

Before we begin, make sure you have the following:

- **Braintrust account and API Key:** Sign up for a free Braintrust account and create an API key (from the Braintrust dashboard settings) for authentication[2]. Export the key in your terminal, e.g. `export BRAINTRUST_API_KEY="YOUR_API_KEY"`.
- **AI Model Access:** An API key for an AI model provider (e.g. OpenAI). In your Braintrust account, configure this key under **AI Providers** (e.g. add your OpenAI key) so Braintrust can route requests[3]. If you plan to use a local model, ensure you have it running and accessible via an API (we'll cover custom providers later).
- **Environment:** This lab uses **Python** (feel free to use Node.js if you prefer – Braintrust has SDKs for both[4]). You should have Python 3.8+ installed on your MacBook (128GB RAM and GPU available, as noted). Create a project directory and initialize a Python virtual environment if desired.
- **Install Packages:** Install the Braintrust SDK and related tools:

```
pip install braintrust autoevals openai
```

This installs the Braintrust client library, the Autoevals library (which provides ready-made scorers and evaluation tools), and OpenAI's SDK (used for model integration)[5][6].

- **Azure Application Insights (optional):** If you want to send telemetry to Azure App Insights, have an Azure subscription and an Application Insights resource ready. We will discuss configuration in the observability section.

Why this matters: Proper setup ensures you can authenticate with Braintrust's platform and access the models needed. Installing the SDKs sets the stage for using Braintrust's

eval and logging APIs. If the API keys or environment are misconfigured, later steps will fail. *Troubleshooting:* If you encounter authentication errors in later steps, double-check that your `BRAINTRUST_API_KEY` and model API keys are set correctly in the environment, and that your Braintrust account has the provider configured.

Step 1: Hello World – Your First Evaluation

In this step, we'll create a simple evaluation to familiarize ourselves with Braintrust's eval framework. We'll define a trivial "agent" that echos a greeting, and use Braintrust to evaluate its output against expected results.

1.1 Define a Simple Eval Script

Create a new file in your project, e.g. **`eval_hello.py`**, with the following content:

```
from braintrust import Eval
from autoevals import Levenshtein

# Define an evaluation called "Say Hi Bot"
Eval(
    "Say Hi Bot",  # Project/experiment name in Braintrust
    {
        "data": lambda: [
            {"input": "Foo", "expected": "Hi Foo"},
            {"input": "Bar", "expected": "Hello Bar"},
        ],
        "task": lambda user_input: "Hi " + user_input,
        "scores": [Levenshtein],
    },
)
```

This eval script uses the Braintrust `Eval()` function to declare an evaluation[7][8]:

- **Data:** Two test cases are provided. Each has an **`input`** (e.g. `"Bar"`) and an **`expected`** output. (Here we intentionally expect "Hello Bar" for the second case to simulate an imperfect response.)
- **Task:** A function that takes the input and returns an output. In our simple agent, the task just prepends `"Hi "` to the input string. In a real app, this might call an LLM or perform a more complex operation.
- **Scores:** A list of scoring functions to evaluate the output. We use the `Levenshtein` scorer from Autoevals, which measures edit distance similarity between the output and expected text. A 100% score means a perfect match[8].

Why it's important: This defines a **baseline evaluation** for an AI behavior. Even a trivial test helps validate your eval setup and demonstrates how Braintrust structures an evaluation (dataset, task, scorers). Defining expected outputs lets us quantitatively measure performance (here, how often the agent says exactly what we expect). If you're

new to evals, this parallels a unit test for AI output – a key practice to catch regressions in AI behavior[9].

Troubleshooting: Ensure you follow naming conventions (Python eval files should be prefixed with `eval_` by default[10]). If your file name or Eval syntax is incorrect, the next step may not discover or run the eval. Also make sure the Braintrust and autoevals libraries are imported correctly; if you get import errors, check that the packages were installed in the right environment.

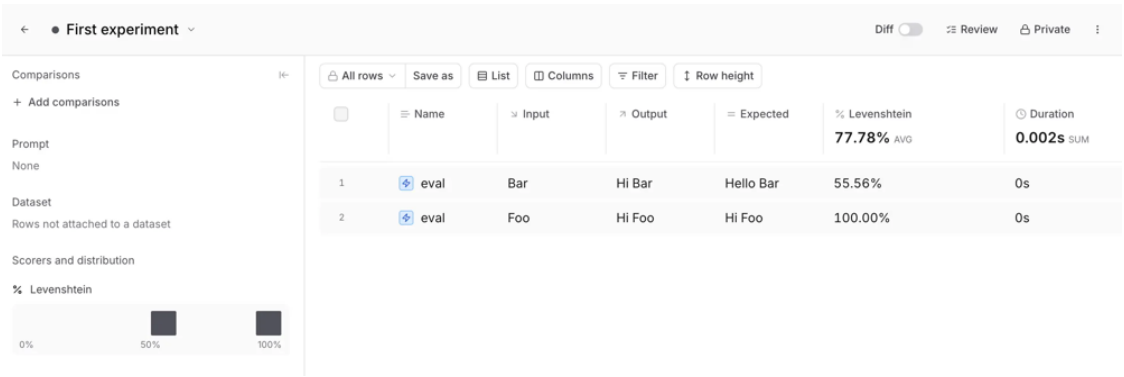
1.2 Run the Evaluation

With the eval script ready, run it via the Braintrust CLI. In the terminal, execute:

```
braintrust eval eval_hello.py
```

The Braintrust CLI will find any evals defined in the file and execute them. You should see output in the console as the eval runs. On completion, the CLI prints a link to the Braintrust web UI for the new experiment[11]. It might also show a summary of results (e.g. average score) in your terminal.

Expected result: The eval should run without errors. Because our `task` returns `"Hi Foo"` for input `"Foo"` (matching expected) but returns `"Hi Bar"` for input `"Bar"` (not matching expected `"Hello Bar"`), we expect a partial success. The Levenshtein scorer will give 100% on the first test and a lower score on the second. In the Braintrust UI, the **Experiment view** for "Say Hi Bot" will show each test case's input, our agent's output, the expected output, and the Levenshtein score[12]. The overall average score might be around ~77.8% in this scenario.



Braintrust experiment result for the first eval. Here we see two test cases (inputs "Bar" and "Foo") with the agent outputs vs. expected. The Levenshtein scores are 55.56% for the "Bar" case (since "Hi Bar" differs from "Hello Bar") and 100% for "Foo" (exact match), averaging ~77.78%. Braintrust's UI highlights these metrics for quick insight.

Why it's important: Running this eval end-to-end confirms that your environment is set up correctly and that Braintrust is logging results. This **baseline test** establishes a starting point. In a real project, you might begin with a simple prompt or function and write a few eval cases to ensure it behaves as expected before scaling up[9].

Troubleshooting: If the CLI command fails, ensure that the `braintrust` CLI is in your PATH. (If you installed via pip, the `braintrust` command should be available. If not, try `python -m braintrust eval ...`). Authentication errors at this stage indicate your `BRAINTRUST_API_KEY` isn't set or is invalid – double-check the environment variable. If the eval runs but you don't see results in the UI, verify that you didn't use the `--no-send-logs` flag (which runs eval locally without sending results to the server)[13].

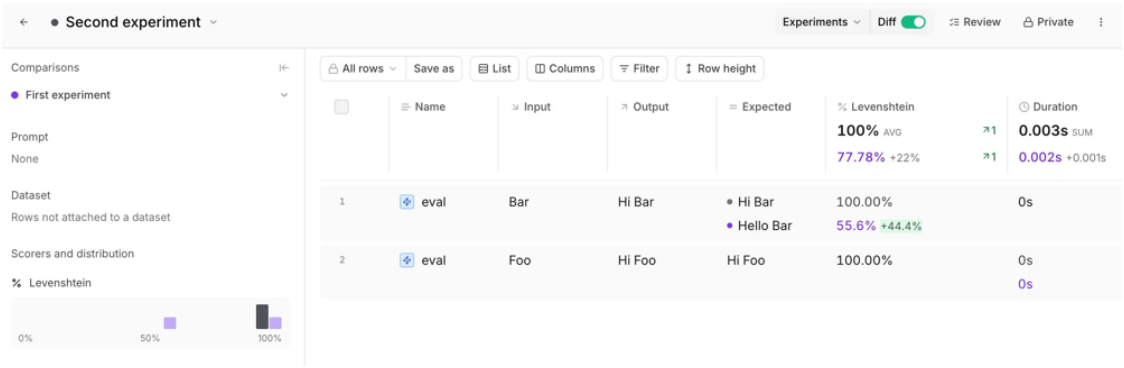
1.3 Interpreting and Improving Results

Open the Braintrust web UI (using the link from the CLI output) to inspect the results. You can see the **score distribution** and each test case outcome. Our initial average was ~77.8%, revealing that the agent's greeting wasn't always as expected.

Let's say we want the agent to greet with "Hello" instead of "Hi" for consistency. We can modify our `task` or expected outputs to improve this score. For example, change the expected output for "Bar" to `"Hi Bar"` to match what the agent actually says *or* change the agent's code to produce `"Hello "` instead of `"Hi "` for the input "Bar". For demonstration, adjust the expected output of the second case to `"Hi Bar"` (so both expected outputs start with "Hi"). Save the file and re-run the eval:

```
braintrust eval eval_hello.py
```

This will record a new experiment (e.g., "Second experiment"). In the UI, you can **compare experiments** to see how the changes affected the score. The second run should now score 100% on both cases (average 100%). Braintrust's UI can display a **diff** showing the improvement over the first run – the Levenshtein score for "Bar" went from ~55.6% to 100%, and the overall average improved by ~22 percentage points.



Improving the evaluation and comparing runs. In the second experiment, the average score is 100% (green), and the diff view (with the first experiment in purple) shows a +44.4% improvement for the "Bar" case's score. This illustrates how Braintrust helps track prompt or code iterations: you get immediate feedback on whether changes improved the agent's performance[14].

Why it's important: Braintrust's evaluation framework enables **iterative prompt/agent development**. By tweaking the agent or prompt and re-running evals, you can quantitatively measure progress towards your quality goals[15][14]. In practice, this

reduces guesswork – you know if a change made things better or worse. Using Braintrust as your "unit test harness" for AI ensures reliability as you build more complex behaviors.

Troubleshooting: If your second experiment still shows no improvement, double-check that your changes were applied. You might have forgotten to save the file or run the eval in the correct directory. Also confirm you are comparing the right experiments in the UI (use the *Comparisons* sidebar to toggle the first experiment on/off).

Step 2: Integrating an LLM via the Braintrust Proxy

Now that we have a basic eval running, let's step up the complexity: use a real Large Language Model (LLM) to perform the task. This will demonstrate Braintrust's ability to integrate with model providers and capture their outputs for evals. We'll use the **Braintrust AI Proxy** to route requests to an actual model (like OpenAI's GPT-3.5 or GPT-4) while automatically logging those calls.

2.1 Why use the Braintrust Proxy?

Braintrust provides an **AI Proxy** that lets you access multiple AI models/providers through a single API endpoint[16]. By pointing your LLM API calls to Braintrust, every request and response is automatically captured for observability and evaluation – no extra logging code needed[17][18]. This means you can swap between providers (OpenAI, Anthropic, local models, etc.) and still leverage Braintrust's evals and monitoring uniformly[19][20]. For our lab, we'll call OpenAI's API via the Braintrust proxy.

2.2 Modify the Task to Call an LLM

Let's replace the simple string concatenation in our eval with an actual model call. Braintrust's Python SDK offers a helper `wrap_openai` to wrap an OpenAI client so that it uses Braintrust as the backend. We'll also adjust the expected outputs to what we anticipate the model will return.

Steps:

1. **Import and initialize the OpenAI client via Braintrust.** At the top of your script (or a new script), add:

```
import os
import openai
import braintrust

# Ensure API keys are set in environment:
# os.environ["BRAINTRUST_API_KEY"] = "YOUR_KEY"  # (optional if already
exported)
openai.api_key = os.environ.get("BRAINTRUST_API_KEY")
openai.api_base = "https://api.braintrust.dev/v1/proxy"
```

Here we configure OpenAI's SDK to use Braintrust's proxy URL as the API base, and we supply our Braintrust API key as the authentication token. This effectively routes any

OpenAI API calls through Braintrust[17]. (Braintrust will use your configured provider key – e.g., OpenAI key – to fulfill the request on its end.)

1. **Update the eval task to use the OpenAI model.** For example:

```
from openai import ChatCompletion

Eval(
    "Say Hi Bot (LLM)",
    {
        "data": lambda: [
            {"input": "Alice", "expected": "Hello, Alice!"},
            {"input": "Bob",   "expected": "Hello, Bob!"}
        ],
        "task": lambda name: ChatCompletion.create(
            model="gpt-3.5-turbo",
            messages=[{"role": "user", "content": f"Say hello to {name}"}]
        )["choices"][0]["message"]["content"],
        "scores": [Levenshtein]
    }
)
```

In this version, the `task` uses OpenAI's chat completion API to generate a greeting. We prompt the model: *"Say hello to {name}"*. The expected outputs are set to `"Hello, Name!"` for each input. When this eval runs, Braintrust's proxy will intercept the `ChatCompletion.create` call and log the request/response, associating it with the "Say Hi Bot (LLM)" experiment.

1. **Run the eval:** `braintrust eval eval_hello.py` (or the new script's name). This will execute the LLM calls. The first run might be a bit slower due to model API latency.

Expected result: The model should produce greetings. For example, GPT-3.5 might respond exactly with `"Hello, Alice!"` or it might include slight variations (like `"Hello, Alice!!"` or adding extra text). The Levenshtein scorer will grade each output against the expected string. Ideally, the model outputs match our expectations closely, yielding high scores. Braintrust will log each LLM call; in the UI, you'll see the experiment with model outputs and scores similarly to before. Additionally, because we used the Braintrust proxy, each completion appears in Braintrust's **trace logs** automatically (more on traces in Step 5).

Why it's important: Integrating a real model shows how Braintrust supports **autonomous agent behavior**. Instead of a canned function, we now have an LLM generating outputs. Braintrust's eval framework remains the same – which underscores its power: you can evaluate any model or agent logic by just swapping the `task` function[21][22]. The proxy integration ensures we don't lose observability when using external AI APIs – every call is captured. This is crucial for debugging AI agent decisions and measuring model performance in a consistent way across providers.

Troubleshooting: Common issues include model API errors. If the OpenAI call fails (e.g., due to invalid API key or reaching usage limits), the eval will error out. Ensure your OpenAI key is correctly added to Braintrust (under AI providers) and not expired. If you get a network error, confirm you have internet connectivity and the Braintrust proxy URL is correct. Also note that the code above expects the OpenAI Python SDK; if you get `AttributeError` on `ChatCompletion.create`, make sure you imported the correct class (`from openai import ChatCompletion`) or use `openai.ChatCompletion.create(...)`. If the model returns unexpected extra text, your expected outputs might not match – adjust the prompt or expected strings accordingly.

2.3 Using Frontier vs. Local Models

Braintrust's proxy isn't limited to OpenAI. It supports many providers out-of-the-box (Anthropic, Azure OpenAI, Hugging Face via HuggingFace Hub, etc.) and **custom providers** for your own endpoints[16][21]. This means you can **pivot between frontier models and local models** easily:

- **Frontier model example:** In the above, we used OpenAI's GPT (a frontier model). We could just as easily switch `model="gpt-4"` (if you have access) or call Anthropic's Claude by using the Anthropic client configured to Braintrust's proxy, etc. No code changes needed besides selecting the model name/provider.

- **Local model example:** Suppose you have a large model running on your machine (for instance, a 70B parameter Llama-based model served via an API). Braintrust can hook into it as a custom provider. In the Braintrust UI, go to **AI Providers -> Add Provider -> Custom**[23]. You'll be prompted to enter:

- A name (identifier for this provider, e.g., "Local-Llama"),
- Model name (how you'll refer to it, e.g., `"llama-70b-local"`),
- Endpoint URL (the HTTP endpoint where your model listens),
- Format (choose the API format it expects: `openai` for OpenAI-compatible APIs, etc.)[24],
- Flavor (`chat` or `completion` depending on model type).

After adding, you can now call your local model via the Braintrust proxy by specifying the model name. For example:

```
ChatCompletion.create(model="llama-70b-local", messages=[...])
```

Braintrust will route this request to your local endpoint. This way, you can run the **same eval** on both a frontier model (e.g., GPT-4) and a local model (e.g., Llama 70B) by just changing the model name, and then compare their performance side by side[16][21].

Expected result: If configured correctly, your local model's outputs will be captured just like the OpenAI ones. You can create a **comparison** in Braintrust UI to see, for instance,

GPT-4's eval vs. your local model's eval on the same dataset[25]. This helps answer questions like "Does the open-source model achieve similar accuracy on my task?"[25].

Why it's important: Braintrust enables **model benchmarking and A/B testing** under a unified framework. For someone like you (with both access to powerful local models and frontier models), this means you can validate if a local model is viable for a product by running the same evals and checking quality differences[25]. It eliminates friction in testing alternatives – all logs and scores are normalized. Moreover, by logging through Braintrust, you maintain observability (input/output records) for *all* models, which is critical when comparing performance or troubleshooting errors.

Troubleshooting: When using custom providers, a common issue is authentication or CORS. Ensure your local model's API is up and reachable from your Mac (check by `curl` to the endpoint). If your custom endpoint requires an auth token, include it in the provider config headers. Braintrust allows templated headers (like `Authorization: Bearer {{api_key}}`) stored securely[26][27]. If something's not working, check the Braintrust **Logs** section – errors might show up there. Also verify the "Format" setting: if your server expects OpenAI-style JSON, use `openai` format to let Braintrust format requests accordingly[28]. Finally, watch out for model-specific quirks: local models might produce different output formats or have slower responses; you may need to adjust timeout settings or prompt formats when switching from a commercial API to a self-hosted model.

Step 3: Building and Evaluating a Multi-Step Agent

Real-world AI agents often involve multiple steps – for example, retrieving information, calling tools/APIs, then formulating a final answer. In this step, we'll construct a simple **prompt-chaining agent** and see how to evaluate and debug it using Braintrust. This showcases Braintrust's ability to trace agent reasoning and evaluate complex workflows, not just single LLM calls.

3.1 Designing a Multi-Step Agent

For illustration, let's consider an agent that plans a trip itinerary (inspired by Braintrust's travel planner example[29]). The agent will perform a few steps for a given user query:

1. **Decision Step:** Use an LLM to decide which action to take (e.g., does the user want flight info, weather info, or both?).
2. **Tool Use Step:** Depending on the decision, call an external "API" (we will simulate with a function) to get data – e.g., call a weather API for a city/date, or a flight API for destinations.
3. **Judge Step:** Use another LLM call to *evaluate* if the tool output is valid or if the agent should try a different action.
4. **Final Step:** Compose the final itinerary answer for the user.

This is a non-trivial agent with branching logic. Such systems can be fragile – errors in early steps can cascade[30]. We need good **observability** to see what each step is doing, and **evaluations** to ensure each part (and the whole) works reliably.

3.2 Implementing the Agent with Logging

We'll write our agent in Python and instrument it with Braintrust. Pseudocode structure:

```
import braintrust
import openai
openai.api_key = os.environ["BRAINTRUST_API_KEY"]
openai.api_base = "https://api.braintrust.dev/v1/proxy"
client = braintrust.wrap_openai(openai.OpenAI(api_key=openai.api_key,
base_url=openai.api_base))
```

Here we wrap the OpenAI client with Braintrust for automatic tracing[31]. Now define some mock tools:

```
import random
from datetime import datetime, timedelta

def mock_weather_api(city, date):
    # Simulate a weather lookup
    return {"forecast": random.choice(["sunny","rainy"]), "date": date,
"city": city}

def mock_flight_api(origin, dest):
    # Simulate a flight search
    return {"price": random.randint(200,800), "origin": origin, "dest": dest}
```

And the agent logic:

```
def plan_trip(query):
    # Step 1: Decide action using LLM
    decision_prompt = f"User query: '{query}'. Should I 'weather', 'flight',
or 'both'?"
    decision = openai.ChatCompletion.create(
        model="gpt-3.5-turbo", messages=[{"role":"user","content":
decision_prompt}]
    )["choices"][0]["message"]["content"].strip().lower()

    # Step 2: Use tools based on decision
    result = {}
    if "weather" in decision:
        city = extract_city(query)  # (assume we have a function to parse
city)
        date = (datetime.now() + timedelta(days=1)).strftime("%Y-%m-%d")
        result["weather"] = mock_weather_api(city, date)
    if "flight" in decision:
        origin, dest = extract_route(query)  # (parse origin/destination)
```

```
        result["flight"] = mock_flight_api(origin, dest)

    # Step 3: Judge step - have LLM verify if result looks reasonable
    judge_prompt = f"Given results {result}, did I choose unnecessary
actions? Reply 'yes' or 'no'."
    judgment = openai.ChatCompletion.create(
        model="gpt-3.5-turbo", messages=[{"role":"user","content":
judge_prompt}]
    )["choices"][0]["message"]["content"].strip().lower()

    # (Agent could use judgment to possibly redo steps; for simplicity, we'll
proceed)

    # Step 4: Final answer composition
    answer = compose_answer(query, result)  # combine info into an itinerary
text
    return answer
```

Each OpenAI call here goes through the wrapped client, so Braintrust logs them. The calls are labeled by Braintrust with metadata like model name, provider, and inputs automatically[32][18]. Additionally, we can insert manual `braintrust.trace()` calls if we want to log custom events or tool outputs:

```
braintrust.trace({
    "name": "decision-step",
    "input": {"query": query},
    "output": {"decision": decision}
})
# ... after getting results:
braintrust.trace({
    "name": "tool-results",
    "input": {"decision": decision},
    "output": result
})
```

This logs structured data for each step, viewable in Braintrust's Trace viewer.

Why log these? For a multi-step agent, it's invaluable to see a **trace of actions** taken. Braintrust will record each `trace` entry and each LLM call as a node in a trace timeline[32][33]. Later, in the Braintrust UI under **Traces**, you can inspect this sequence to debug how the agent arrived at its final answer.

3.3 Evaluating the Agent's Performance

To evaluate this agent, we can treat `plan_trip(query)` as our task function in an eval. For instance, define a dataset of user queries and expected outcomes or properties:

```
eval_data = [
    {"input": "Plan a trip from NYC to London next week", "expected": "Flight
and weather info"},
```

```
    {"input": "Do I need an umbrella in Paris tomorrow?", "expected":
"Weather info only"},
    # ... more scenarios
]
```

Instead of exact string matches (which would be brittle for complex outputs), we might use a custom scorer. For example, we could write a scorer that checks the agent's answer for presence of certain keywords (ensuring the agent included flight info when expected, etc.), or use an LLM-based scorer for qualitative evaluation (Braintrust's Autoevals has an `LLMClassifier` or `Factuality` scorer that might be applicable[34][35]).

Setting up an eval might look like:

```
from autoevals import LLMClassifier

Eval(
  "Travel Agent E2E",
  {
    "data": lambda: eval_data,
    "task": plan_trip,
    "scores": [LLMClassifier(lambda output, expected:
                f"Does this answer include {expected}? Reply yes or no.")]
  }
)
```

Here we use `LLMClassifier` (which uses an LLM to judge a condition) to verify if the agent's answer contains the expected type of info. This is an example of **model-graded evaluation** – useful for subjective or complex criteria[36].

Run this eval with `braintrust eval ...` as before. Braintrust will execute `plan_trip` for each input, logging every step's trace, and then score the final answers.

Expected result: You will get an experiment "Travel Agent E2E" with results for each test query. The scores might be simply 100% or 0% if using a yes/no classifier. More importantly, you now have **rich trace data** for each eval run. In the Braintrust UI, go to **Traces** or the experiment details: you can click on an eval run and see the **entire chain of calls** the agent made (the decision LLM call, the weather API call, the judge call, etc.), since we instrumented the agent with Braintrust. Each iteration (each data row) is traced individually[37][38], giving deep insight into the agent's reasoning and where things might go wrong.

For example, if one scenario shows the agent made an unnecessary API call, you might see in the trace that the `judge_step` output was "yes" (indicating it took an unnecessary action) when it shouldn't have. That would highlight a flaw in the decision logic. You could then adjust the agent's prompting or logic and re-run to see if it improves (just like we did in Step 1).

Why it's important: Complex agents can **fail in unpredictable ways[39]**, and small errors can compound over multiple steps[30]. Braintrust's tracing and eval tools let you *isolate* problems. You can evaluate the entire end-to-end agent (did it produce a correct final answer?), and also zoom into specific steps (did the judge step correctly identify a bad action?). This dual approach helps ensure reliability before deploying the agent. By logging everything, you create a feedback loop: evaluation data (traces and results) show where the agent needs improvement, and you can refine it systematically.

Troubleshooting: Building multi-step agents is challenging. If your agent function crashes (perhaps due to a bug in parsing or a missing key), the eval run for that data point will error – Braintrust usually records the error in the output. Check the Braintrust **Logs** for error messages or stack traces. If traces aren't appearing, ensure you called `braintrust.login()` or otherwise initialized logging (in our code, wrapping the OpenAI client via `wrap_openai` implicitly handles a lot of this[40], but calling `braintrust.initLogger(project_name="Travel Agent E2E", apiKey=...)` is another way[41]). If the `LLMClassifier` scorer yields inconsistent results, consider simplifying the prompt it uses or switch to a more deterministic scorer for testing. Always double-check that your expected output logic in the eval is correct – it's easy to mistakenly mark a good output as bad if your scorer is off.

Step 4: Enabling Observability with OpenTelemetry and Azure Monitor

With our agent up and running, the next focus is **Observability**: monitoring the agent in real-time, instrumenting it according to standards, and exporting telemetry to external systems like Azure Application Insights. Braintrust provides robust observability features out-of-the-box and integrates with OpenTelemetry (OTel), an open standard for telemetry data.

4.1 Why Observability Matters for AI Agents

Traditional software observability (logs, metrics, traces) is critical for debugging, and AI systems are no exception. In fact, AI agents introduce non-determinism, making observability even more vital – you need to monitor not just performance (latency, errors) but also the quality of decisions and outputs[42]. Telemetry from your agent serves two purposes:

- **Monitoring & Debugging:** Find and fix issues, track latency, ensure the agent is calling the right tools, etc., in production.
- **Feedback for Improvement:** Use logs of real interactions as new eval data to continuously improve the agent[43].

However, AI observability has been a fragmented space with various vendors and frameworks. The OpenTelemetry community is defining **semantic conventions for generative AI** to standardize how we emit traces/logs from LLMs and agents[44][45]. Aligning with these standards ensures your telemetry can be interpreted uniformly across tools and avoids vendor lock-in on proprietary formats[44].

Braintrust embraces these principles by allowing you to **treat Braintrust as an OTel backend** – meaning you can send OTel data to it – and by exposing the logs/traces of LLM calls in an open-friendly way[46]. We'll leverage that, and also forward data to Azure for a familiar APM view.

4.2 Instrumenting with OpenTelemetry (OTel)

If you have an application or agent that already uses OTel instrumentation (for example, the Vercel AI SDK or LangChain instrument their calls), you can direct those spans to Braintrust. In our case, we manually logged via Braintrust's SDK, but let's also ensure OTel is wired so we can send data elsewhere (Azure).

Using Braintrust as an OTel collector: Braintrust provides an OTLP endpoint (OpenTelemetry Protocol) that you can set as a trace exporter. Simply set the following environment variables in your app environment or `.env` file:

```
OTEL_EXPORTER_OTLP_ENDPOINT=https://api.braintrust.dev/otel
OTEL_EXPORTER_OTLP_HEADERS=Authorization=Bearer <YOUR_BRAINTRUST_API_KEY>, x-bt-parent=project_name:<Your Project Name>
```

These variables, as described in Braintrust's docs[47], tell the OTel SDK/Collector to send all telemetry to Braintrust's endpoint, with your API key for auth. The special `x-bt-parent` header tags the trace data with a specific project or experiment – you can use your Braintrust project name or ID here so the traces appear in the right place in the UI[48]. Once this is set, any OTel spans created in your code will be forwarded to Braintrust.

For example, if using the Vercel AI SDK in a Next.js app, you'd configure the OTel provider with Braintrust's endpoint and enable their instrumentation hook[47][49]. In our Python scenario, if we had an OTel tracer, we could instantiate it to capture spans around our agent's steps and OTel would send them to Braintrust.

OpenTelemetry semantic conventions: To abide by the GenAI semantic conventions, ensure your spans and logs use the recommended attributes. For instance, when logging an LLM call, include attributes like `ai.model.id` (model name), `ai.request.prompt` (prompt text), `ai.response.content` (model output), etc., as defined in the spec. If you use Braintrust's `trace()` or proxy, much of this is handled. But if doing custom OTel spans, you can set attribute keys per the emerging standard (refer to the latest OpenTelemetry spec for GenAI spans) – this ensures compatibility with tools that understand these conventions.

Expected result: After configuring OTel, run your agent (for example, through the eval or in a live app). You should not notice any difference in functionality, but behind the scenes, OTel will emit spans. Braintrust will start receiving those spans as well. In the **Braintrust UI**, under **Traces**, you'll see trace data streaming in corresponding to your operations (each LLM call, each tool invocation if instrumented, etc.). The data will include standard fields and your custom attributes, making it easier to search and filter (e.g., you can filter

traces by `model:gpt-3.5-turbo` or similar). Essentially, this duplicates what we did with `braintrust.trace()` but via a standardized pipeline.

Why it's important: Using OTel brings your AI observability into the same ecosystem as the rest of your app's telemetry. It means you can use open-source or third-party tools to analyze the data if needed, and you adhere to best practices (ensuring that, for example, any dashboard or APM tool that knows about the GenAI semantic conventions can readily interpret your trace data). Braintrust acting as an OTel backend is a huge plus – it **simplifies logging from many environments** (you don't have to use Braintrust SDK in every language; any OTel instrumentation can connect)[46]. Furthermore, it future-proofs your observability: as standards evolve, you can emit new fields and know Braintrust (and Azure, etc.) will handle them.

Troubleshooting: If you don't see any traces in Braintrust after enabling the OTel exporter, a few things to check: - Ensure the OTel SDK is initialized in your application. For Python, you might need to install `opentelemetry-sdk` and set up a tracer provider and exporter in code if not already done. - Verify that your environment variables are actually being read by the app. - The `Authorization` header should have the word `Bearer` followed by your API key – missing this format or a typo in the key will prevent ingestion[48]. - Also, OTel typically batches and exports periodically; ensure the app runs long enough or force a flush of spans on shutdown so that they get sent to Braintrust. - You can test with a simple manual span to see if it appears in Braintrust logs. If problems persist, Braintrust's documentation and community (Discord) can be helpful, as well as OTel's own docs for setting up exporters.

4.3 Sending Telemetry to Azure Application Insights

Finally, we want to send our telemetry to Azure Application Insights, so our organization's monitoring tools can also observe the AI agent's behavior. Azure Monitor has native support for OpenTelemetry – via the Azure Monitor OpenTelemetry Exporter or Distro for various languages[50]. We will configure an exporter to send data to Application Insights in addition to Braintrust.

Configure Azure Monitor exporter: You'll need the instrumentation key (or connection string) for your Application Insights resource. For Python, install Azure's exporter:

```
pip install opentelemetry-exporter-azuremonitor
```

Then, in your code, after setting up the OTel provider for Braintrust, also add:

```
from opentelemetry.exporter.azuremonitor import AzureMonitorTraceExporter
from opentelemetry.sdk.trace import TracerProvider
from opentelemetry.sdk.trace.export import BatchSpanProcessor

# Set up a tracer provider (if not already done by your framework)
trace.set_tracer_provider(TracerProvider())
```

```
# Configure Azure exporter
azure_exporter = AzureMonitorTraceExporter(
connection_string="InstrumentationKey=<YOUR_KEY>" )
trace.get_tracer_provider().add_span_processor(BatchSpanProcessor(azure_expor
ter))
```

This attaches an exporter that will send spans to Azure. Ensure this runs at startup. (In a web app, you might use the Azure Monitor OTel Distro which simplifies some of this[51].)

Now your telemetry data will be **forked**: it goes to Braintrust's OTLP endpoint *and* to Azure. In Application Insights, you can open the **Live Metrics** or **Transaction Search** to see incoming traces. You should see spans corresponding to your agent's operations. For example, each LLM call might show up as a dependency or custom event with properties like `ai.model.id`, etc., depending on how the exporter sends them. You can build dashboards or alerts in Azure Monitor just as you would for other application telemetry.

Expected result: In Azure Application Insights, the telemetry from your agent (traces, perhaps custom metrics if you record any) will start appearing. You can query the logs for specific attributes (e.g., in Log Analytics, run queries on custom properties for your GenAI spans). You'll also continue to have the data in Braintrust. This dual-telemetry setup means Braintrust serves for AI-specific analysis (comparing quality, looking at traces with the rich context of evals and datasets), while Azure Monitor provides a centralized view alongside other system metrics (CPU, memory, etc.) and leverages Azure's alerts and visualization. Both are useful: for instance, you might set up an **Azure alert** if response latency goes above a threshold, and a **Braintrust alert** if quality score drops or a certain error rate spikes[52].

Why it's important: Many enterprises use Azure's monitoring for all their apps. By sending AI telemetry there, you **integrate AI agents into existing DevOps workflows**. Engineers and product managers can use familiar tools (Application Insights, dashboards, etc.) to track the AI component. Moreover, using OpenTelemetry to do this means you're not locked into one monitoring solution – the data is in a standard format. This aligns with the best practice of avoiding siloed telemetry[44]. You get the best of both worlds: Braintrust for AI-focused evals/observability, and Azure for holistic monitoring. Also, Azure's analytics might catch things like memory leaks or dependency failures that are outside Braintrust's scope.

Troubleshooting: Getting OTel data into Application Insights can be tricky if the configuration is off. If you don't see data in Azure, double-check your connection string or instrumentation key, and ensure no firewall is blocking outgoing data. Azure's exporter may have its own batching delay – check again after a few minutes. The Azure Monitor Distro can simplify setup by automatically collecting standard metrics and integrating with the Azure pipeline[51]. If you see data in Azure but it's not labeled clearly, you might need to add custom dimensions or use Azure's parsing to pull out the `ai.*` fields. Consult Azure's docs on OpenTelemetry mapping to App Insights fields[53] – some attributes might appear under customDimensions in the logs.

Step 5: Continuous Evaluation and CI/CD Integration

The final step of our lab is to ensure all this work (our evals and agent quality checks) becomes part of a continuous integration/continuous deployment (CI/CD) pipeline. We want that every time we update our agent (code or prompts), evaluations run automatically to catch regressions before shipping changes to production. Braintrust provides a **GitHub Action** to run evals in CI easily[54].

5.1 Setting Up GitHub Actions for Evals

Assuming you have your project in a GitHub repository, you can add a workflow for CI. Braintrust's GitHub Action (`braintrustdata/eval-action`) can execute your eval scripts on each pull request or push, then report the results as a comment on the PR[55].

Steps:

1. In your repo, create a workflow file (e.g., `.github/workflows/run-evals.yml`).
2. Use the following template (from Braintrust docs):

```
name: Run evals on PRs
on: pull_request

jobs:
  eval:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v4
        with:
          python-version: "3.10"

      - name: Install dependencies
        run: pip install braintrust autoevals openai  # and any other deps

      - name: Run Evals
        uses: braintrustdata/eval-action@v1
        with:
          api_key: ${{ secrets.BRAINTRUST_API_KEY }}
          runtime: python
          root: .
```

This workflow triggers on pull requests. It checks out the code, sets up Python, installs needed packages, then invokes the Braintrust eval action. We provide our Braintrust API key securely via GitHub Secrets, and specify `runtime: python` (since our evals are in Python)[56][57]. The `root` is set to `.` meaning look for eval files in the repository root (adjust if your eval scripts are in a subfolder).

Note: Braintrust's action supports both Node and Python projects by setting `runtime` and the appropriate package manager[58]. It can also cache results and only run affected evals, but the above is a basic setup.

1. Commit this workflow to your repo. On a new pull request (or push), the action will run. It uses `braintrust eval` under the hood to run all evals, then posts a summary comment on the PR with the results[59][60].

Expected result: When you open a pull request that changes your agent or evals, you should see a GitHub Actions check running the evals. On completion, a comment (from **github-actions** bot) will appear on the PR showing something like:

Example GitHub PR comment from Braintrust evals. In this report, you see the name of the eval (e.g. "Say Hi Bot") with a link to the detailed results, and a table of scores with their averages, improvements, and regressions compared to the baseline[61]. Green/yellow/red indicators quickly show if any metric improved or regressed. For instance, "Levenshtein 85% (+1pp)" means the average Levenshtein score is 85%, up 1 percentage point from the base, with some test cases improved (green) and some regressed (red).

This automated report provides an immediate quality gate. If a change causes the agent's performance to drop (e.g., more regressions than improvements, or a critical metric dips), the team can catch it during code review **before** merging. You can even configure the action to fail the workflow for certain regressions if desired (using the `terminate_on_failure` option or custom scripts)[62].

Why it's important: Incorporating evals into CI/CD brings AI development in line with standard software engineering practices. It's like running unit and integration tests on each commit – but for AI behavior. This ensures you **catch issues early**: for example, if a change to prompt or code suddenly makes the agent respond incorrectly to some test cases, you'll know immediately rather than finding out from user feedback. As Braintrust notes, advanced teams integrate the GitHub action to run evals on every PR, preventing quality regressions from ever reaching production[63]. It promotes a culture of data-driven improvements: every code change can be tied to an eval impact (did our reliability metric go up or down?). Over time, these evals and their historical trends become a key part of your deployment decisions (you might block a release if critical evals fall below a threshold).

Troubleshooting: Ensure the GitHub Action is set up with the correct secrets and parameters: - The `BRAINTRUST_API_KEY` secret must be added in your repo's Settings > Secrets. - Make sure the workflow YAML indentation and syntax are correct (GitHub will show an error in Actions if the file is misconfigured). - If the action runs but doesn't post a comment, check that the workflow has `pull-requests: write` permission as in the example[64][65] – without this, it cannot comment on the PR. - If you have many evals or they call external APIs, be mindful of timeouts and API rate limits in the CI environment. You might need to set `max_concurrency` in evals to avoid hitting rate limits[66], or use

`use_proxy: true` (the default) to leverage Braintrust's caching of identical LLM calls[67]. - Finally, if you want to surface these results more formally, you could fail the action on regressions by parsing the output or use Braintrust's API to fetch scores and set conditions. The current action doesn't auto-fail on regressions (it just comments), so for strict gating, implement an extra step to decide pass/fail based on the reported metrics.

Conclusion

In this lab, we journeyed from a simple "Hello World" evaluation to a full-fledged AI agent with observability and continuous evaluation:

- We learned how Braintrust provides a unified platform to **iterate, evaluate, and monitor** AI systems, making development more systematic and reliable[1][9].
- Starting with basic evals, we saw the value of defining expected outcomes and automatic scorers to quantify an agent's performance, catching issues like inconsistent responses early.
- We integrated a real model via Braintrust's proxy, demonstrating seamless swapping between **frontier models and local models** while retaining logging and evaluation capabilities[16][21].
- We built a multi-step agent and used Braintrust's **tracing** to illuminate each decision and action the agent took, which is crucial for debugging complex AI behavior. By evaluating both end-to-end outcomes and individual steps, we can pinpoint weaknesses and iteratively improve the agent[30][29].
- We enabled **observability** by configuring OpenTelemetry, aligning our instrumentation with emerging industry standards for AI telemetry[44]. We also showed how to forward this data to Azure Application Insights, combining Braintrust's AI-focused monitoring with Azure's broad APM capabilities for a holistic view of the system.
- Finally, we integrated Braintrust evals into a CI/CD pipeline via GitHub Actions[54]. This closes the loop by ensuring every change is validated against our evaluation suite, preventing regressions and giving the team confidence in the quality of the AI agent at each iteration.

By following this lab, you have a template for robust AI agent development: **design, eval, observe, and repeat**. As you continue, you can expand your eval datasets (Braintrust allows managing datasets and even human-in-the-loop review[68]), introduce more complex scorers (like bias or toxicity checks), and set up Braintrust **Monitors/Alerts** to watch your production metrics and trigger notifications or automations when something goes off track[69].

Remember that building reliable AI is an ongoing process. Braintrust's platform, combined with open standards like OpenTelemetry, gives you the tools to make this process measurable and transparent. With systematic evals and rich telemetry, you can move fast

with your AI projects **without sacrificing quality or trust**. Happy building, and may your agents always behave!

Sources:

- Braintrust Documentation and Cookbook – official guides on evals, agents, providers, and observability[7][21][47][54].
- OpenTelemetry Specification – semantic conventions for generative AI, guiding how we instrument AI apps[44].
- Azure Monitor Documentation – details on exporting OpenTelemetry data to Application Insights[50].

---

[1] [68] Get started - Docs - Start - Braintrust

https://www.braintrust.dev/docs/start

[2] [4] [7] [8] [10] [11] [12] [13] [14] Eval via SDK - Docs - Start - Braintrust

https://www.braintrust.dev/docs/start/eval-sdk

[3] [5] [6] [34] [35] Prompt versioning and deployment - Docs - Braintrust

https://www.braintrust.dev/docs/cookbook/recipes/PromptVersioning

[9] [15] [39] [52] [69] Braintrust - The evals and observability platform for building reliable AI agents

https://www.braintrust.dev/

[16] [19] [20] [22] AI providers - Docs - Braintrust

https://www.braintrust.dev/docs/providers

[17] [18] [21] [23] [24] [26] [27] [28] [32] [33] [41] Custom providers - Docs - Braintrust

https://www.braintrust.dev/docs/providers/custom

[25] [46] [47] [48] [49] Using OpenTelemetry for LLM observability - Docs - Braintrust

https://www.braintrust.dev/docs/cookbook/recipes/OTEL-logging

[29] [30] [31] Evaluating a prompt chaining agent - Docs - Braintrust

https://www.braintrust.dev/docs/cookbook/recipes/PromptChaining

[36] AutoEvals is a tool for quickly and easily evaluating AI model outputs ...

https://github.com/braintrustdata/autoevals

[37] Building reliable AI agents - Docs - Braintrust

https://www.braintrust.dev/docs/cookbook/recipes/AgentWhileLoop

[38] [40] An agent that runs OpenAPI commands - Docs - Braintrust

https://www.braintrust.dev/docs/cookbook/recipes/APIAgent-Py

[42] [43] [44] [45] AI Agent Observability - Evolving Standards and Best Practices | OpenTelemetry

https://opentelemetry.io/blog/2025/ai-agent-observability/

[50] [51] Enable OpenTelemetry in Application Insights - Azure Monitor | Microsoft Learn

https://learn.microsoft.com/en-us/azure/azure-monitor/app/opentelemetry-enable

[53] Application Insights OpenTelemetry data collection - Azure Monitor

https://learn.microsoft.com/en-us/azure/azure-monitor/app/opentelemetry-overview

[54] [55] [59] [60] [64] [65] [66] Run evaluations - Docs - Guides - Experiments - Braintrust

https://www.braintrust.dev/docs/guides/experiments/run

[56] [57] [58] [62] [67] Braintrust eval · Actions · GitHub Marketplace · GitHub

https://github.com/marketplace/actions/braintrust-eval

[61] braintrust.dev

https://www.braintrust.dev/_next/image?url=%2F_next%2Fstatic%2Fmedia%2Fgithub-actions-comment.f49b3e26.png&w=3840&q=75

[63] Webinar recap: Eval best practices - Blog - Braintrust

https://www.braintrust.dev/blog/best-practices